CI/CD Pipeline Implementation using Jenkins with Maven & Gradle

1. Introduction

Continuous Integration (CI) is a software development practice where code changes are automatically built and tested. In this experiment, we implement a CI pipeline using Jenkins integrated with Maven and Gradle build tools.

2. Objective

- To configure Jenkins for CI automation
- To integrate GitHub with Jenkins
- To build and test applications using Maven and Gradle
- To execute automated pipelines using Jenkinsfile

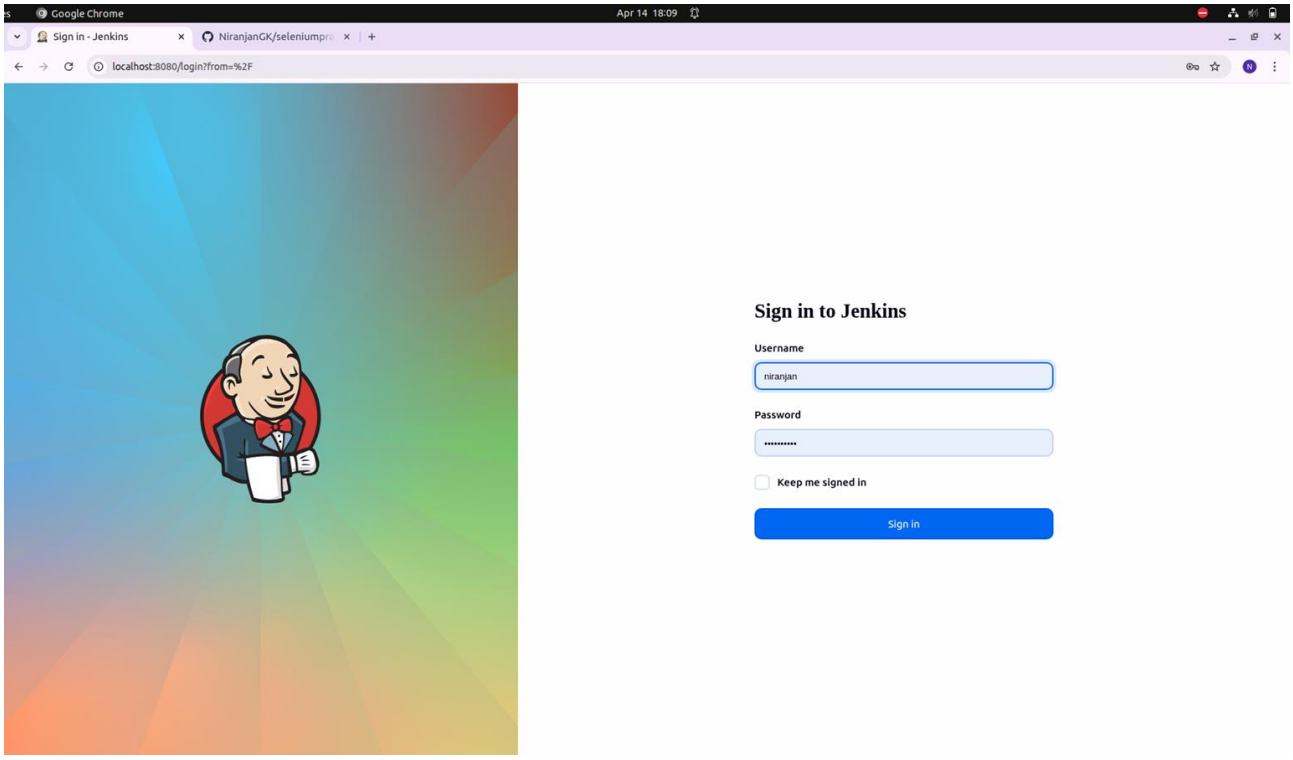
3. Tools & Technologies Used

- Jenkins
- GitHub
- Maven
- Gradle
- Java (JDK 11 or above)

4. IMPLEMENTATION

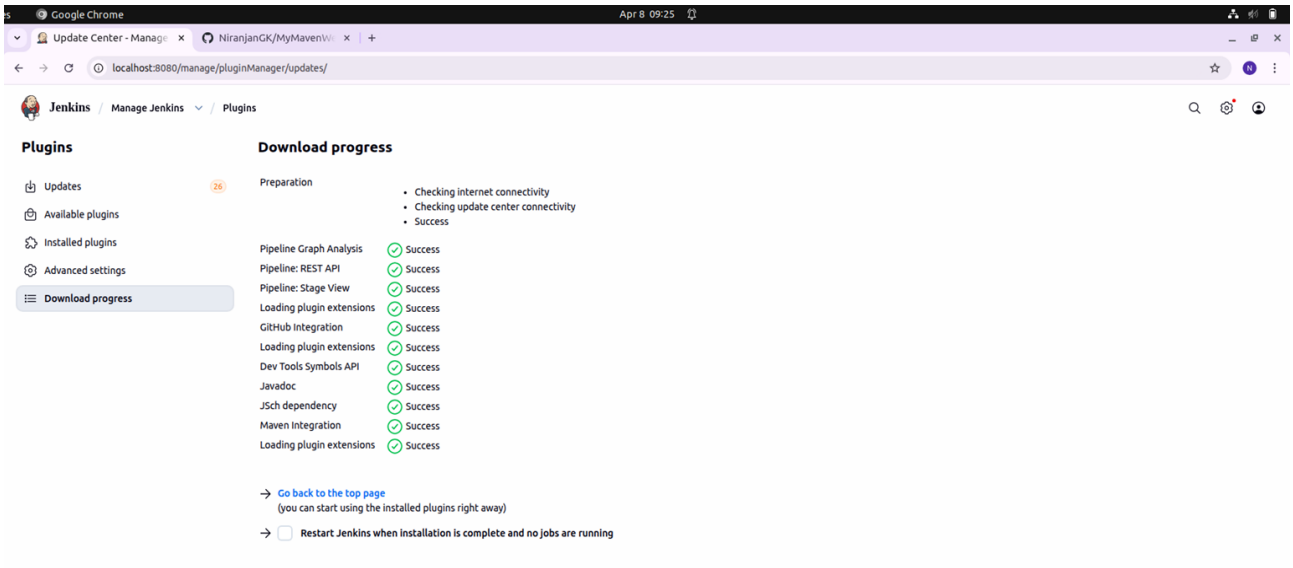
Phase 1: Jenkins Setup

- Access Jenkins dashboard
- Login to system



Phase 2: Plugin Configuration

- Install required plugins:
    - Maven Integration
    - GitHub Integration
    - Pipeline Stage View

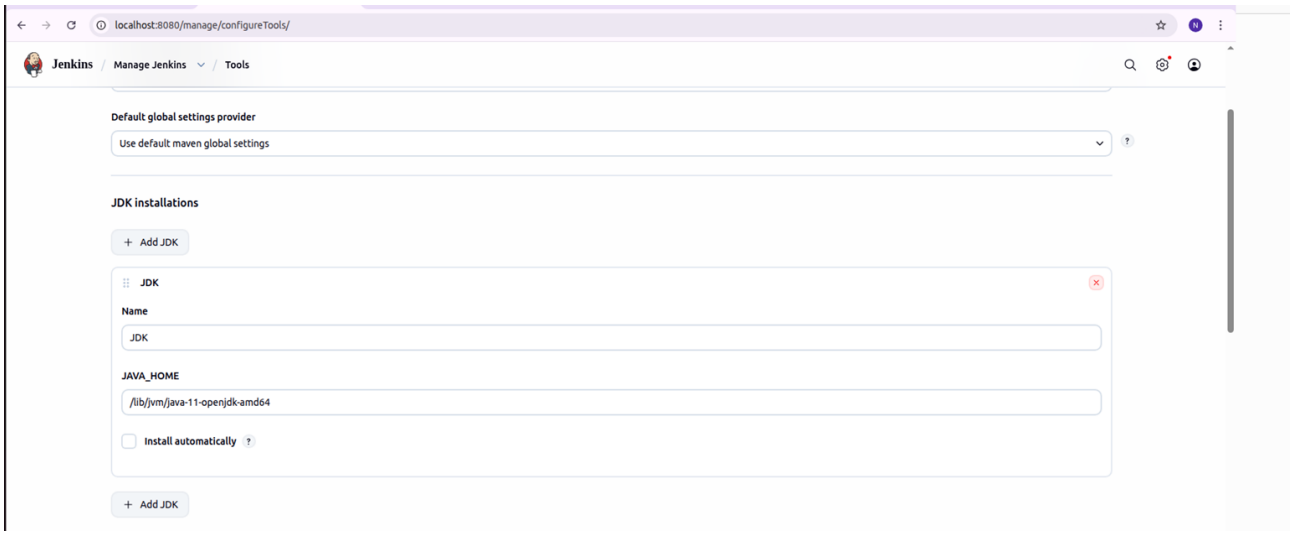


Phase 3: Tool Configuration

- Configure:
    - JDK
    - Maven
    - Git
    - Gradle

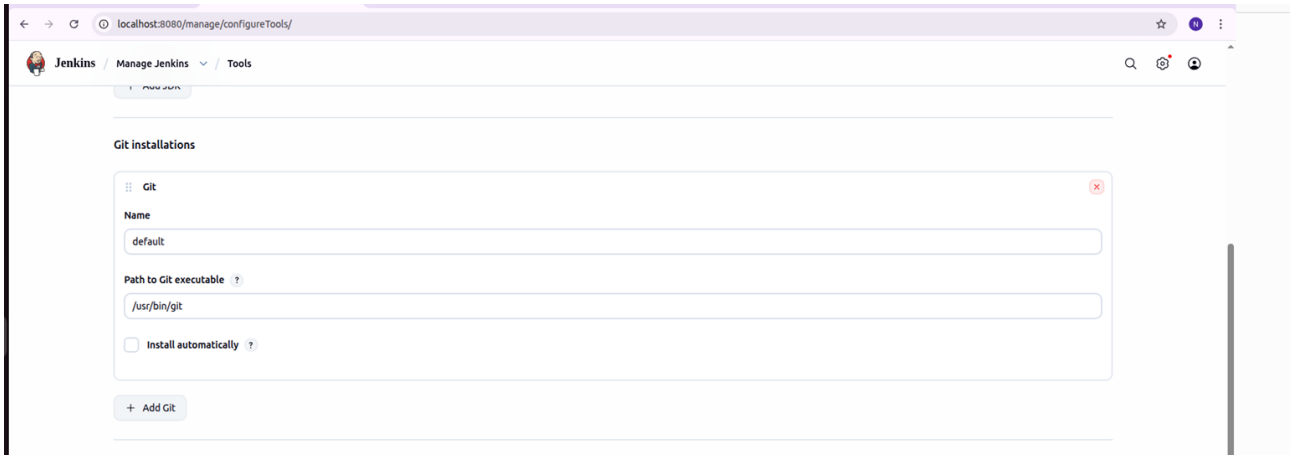
Navigate to **Manage Jenkins →Tools**

Configure **JDK** installations:



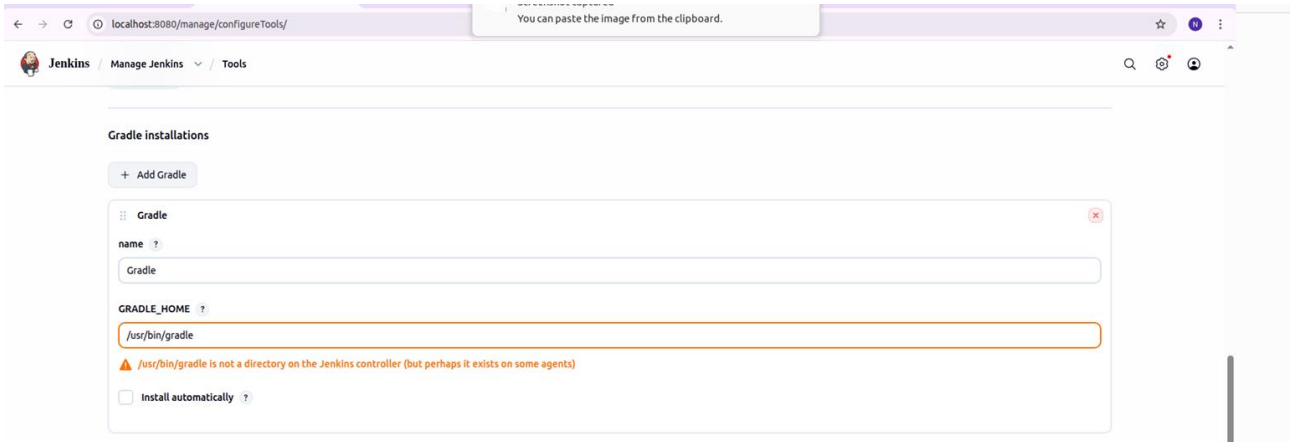
Navigate to **Manage Jenkins →Tools**

Configure **Git** installations:



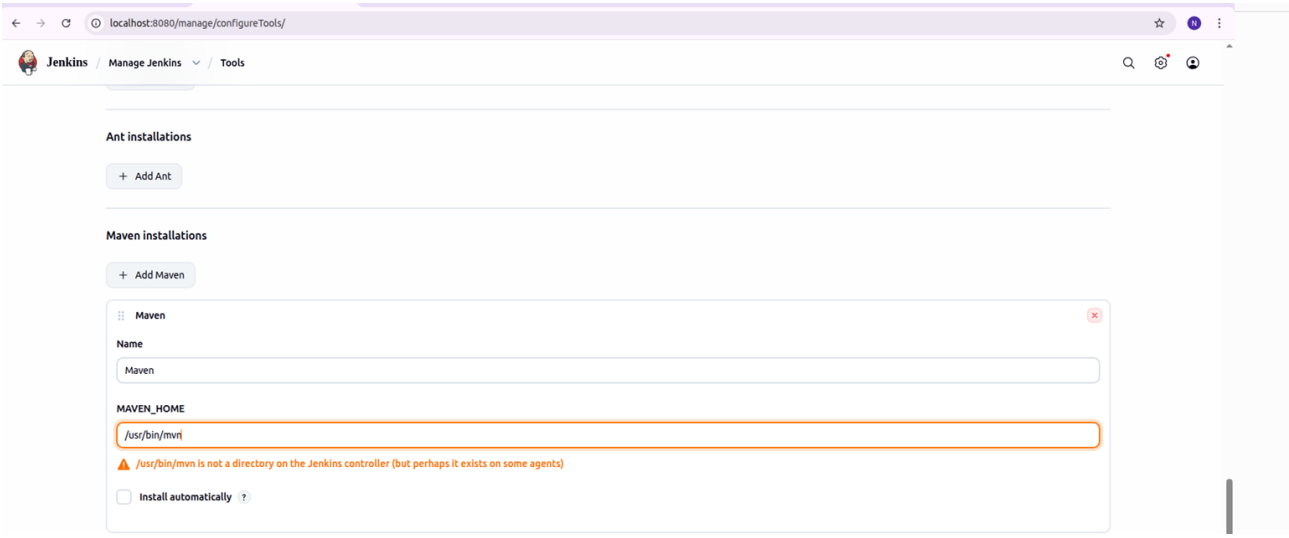
Navigate to **Manage Jenkins →Tools**

Configure **gradle** installations:



Navigate to **Manage Jenkins →Tools**

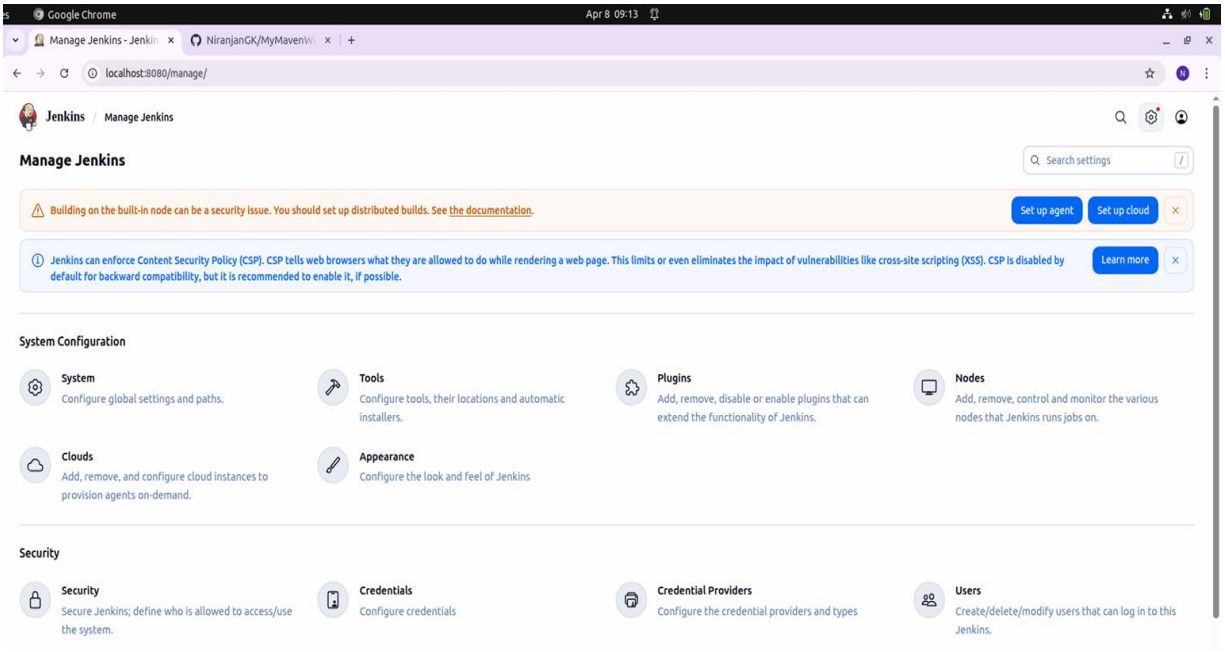
Configure **Maven** installations:



Phase 4: Credentials Setup

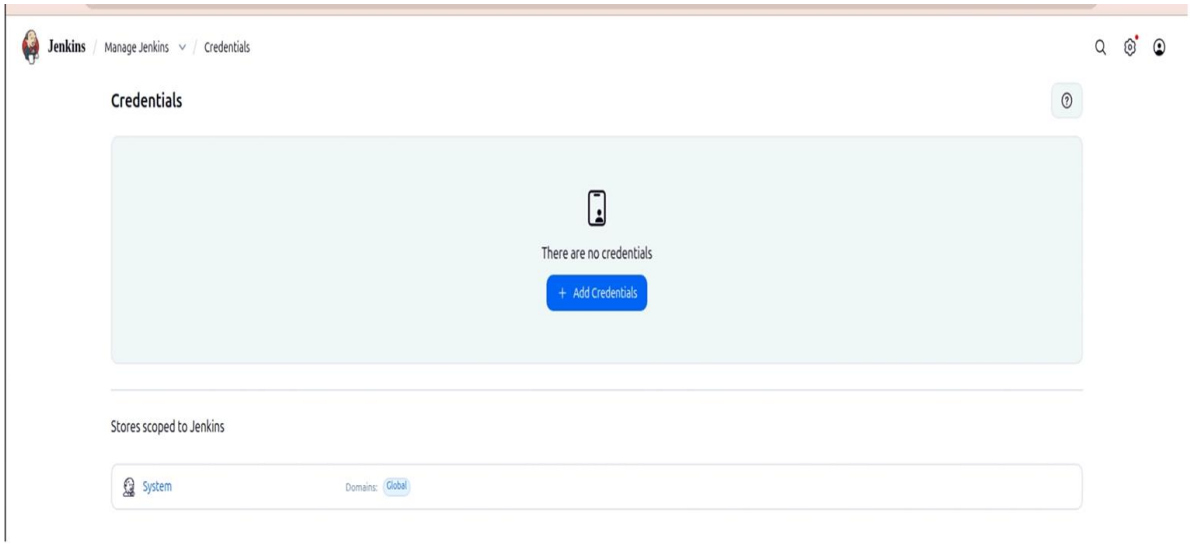
- Add GitHub credentials in Jenkins

Navigate to **Manage Jenkins → Credentials**

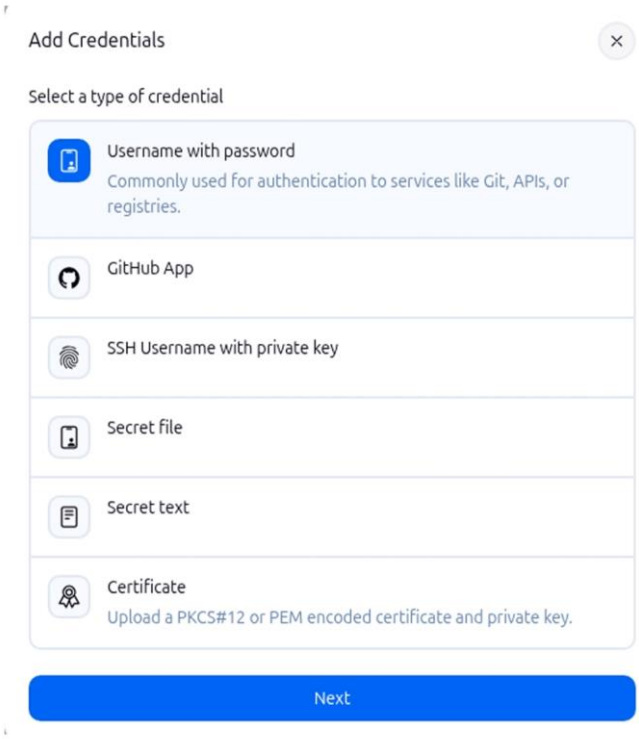


click on **Add Credentials:**

Add new credentials and select the type of credentials as **"Username with Password"** which is



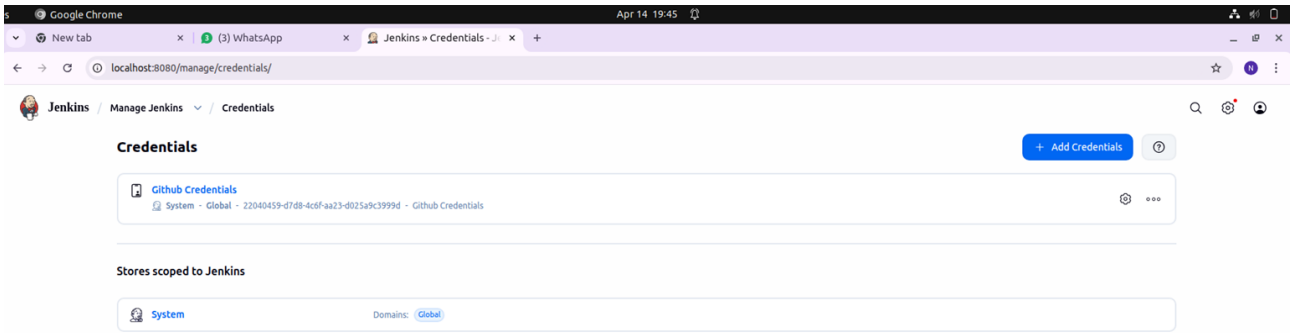
used for authentication to services like Git,APIs



Click on **Next**:

Add **Username** and **Password** and click on Create:

Now the Credential is successfully created and it is available in Jenins pipeline.



Jenkins Plugin Installation and Configuration

1. Introduction

Jenkins is a widely used automation server that supports Continuous Integration and Continuous Deployment (CI/CD). Its functionality can be extended using plugins, which allow integration with various tools and technologies such as GitHub, Maven, and pipeline visualization tools.

2. Objective

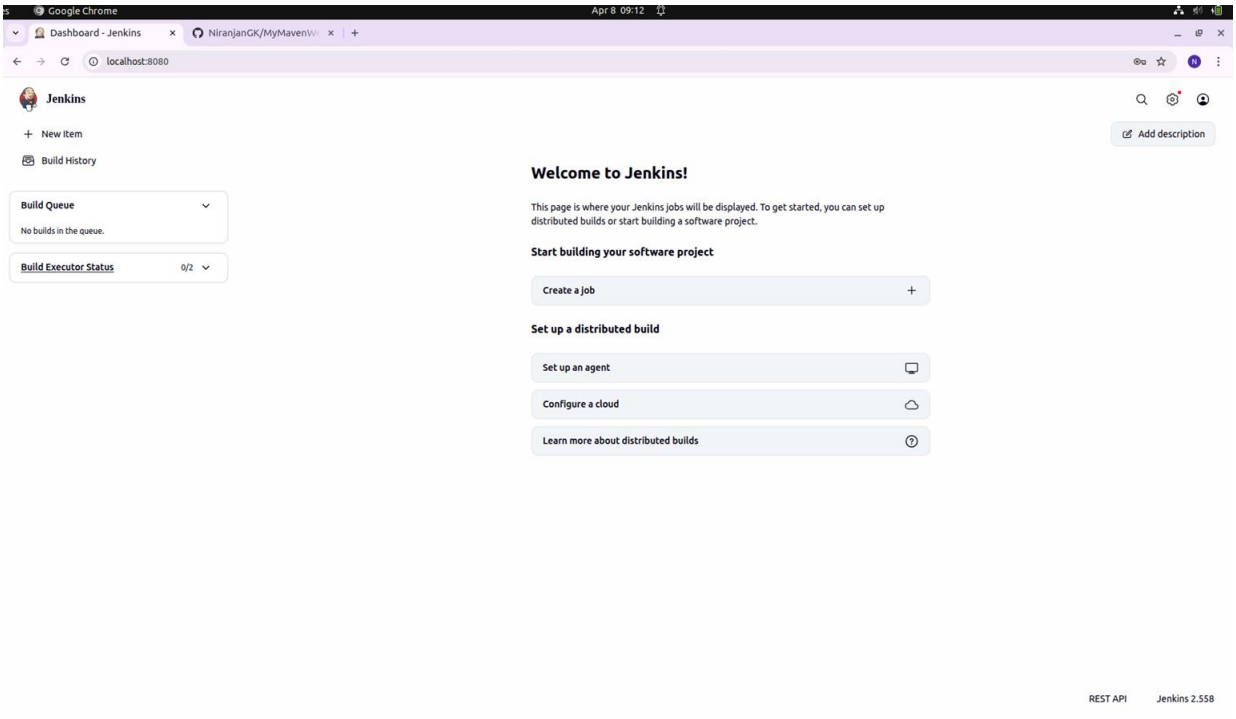
- To install essential Jenkins plugins
- To enable integration with external tools
- To enhance pipeline visualization and execution
- To prepare Jenkins for CI/CD pipeline creation

3. Prerequisites

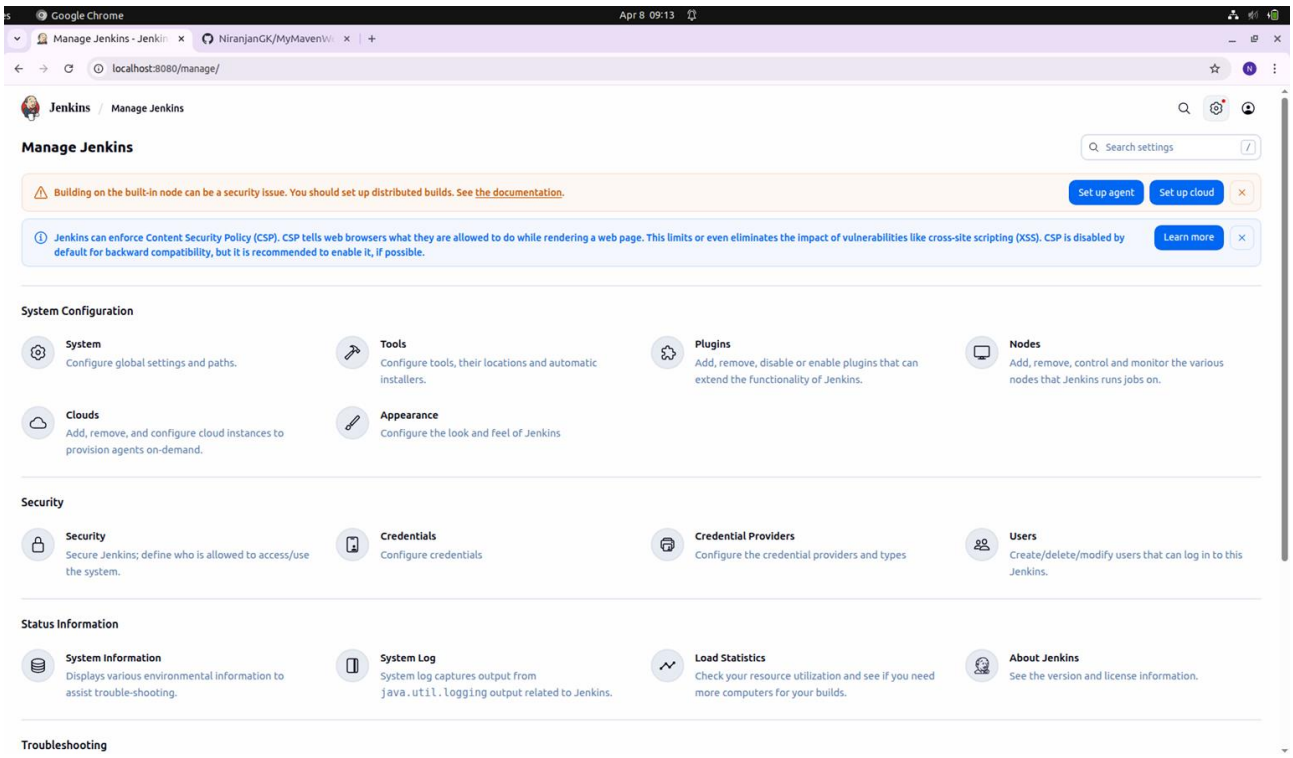
- Jenkins installed and running
- Internet connection
- Admin access to Jenkins dashboard

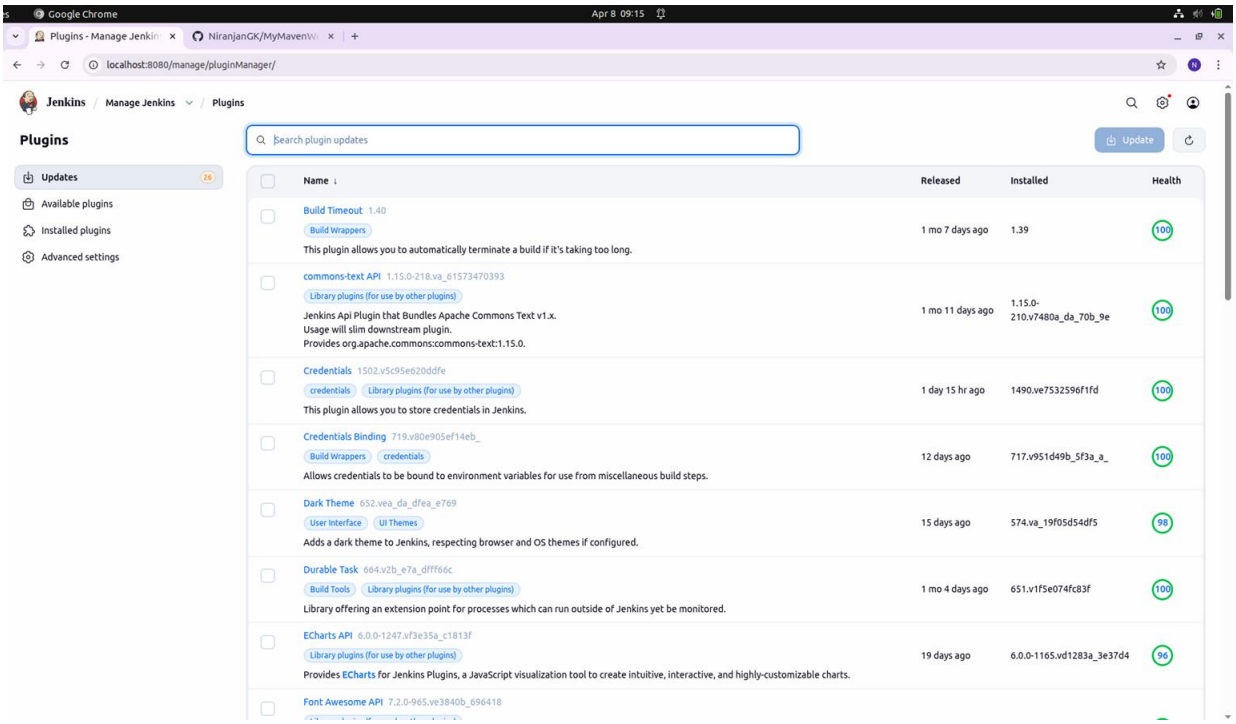
4. Steps to Install Plugins in Jenkins

1. Open Jenkins Dashboard

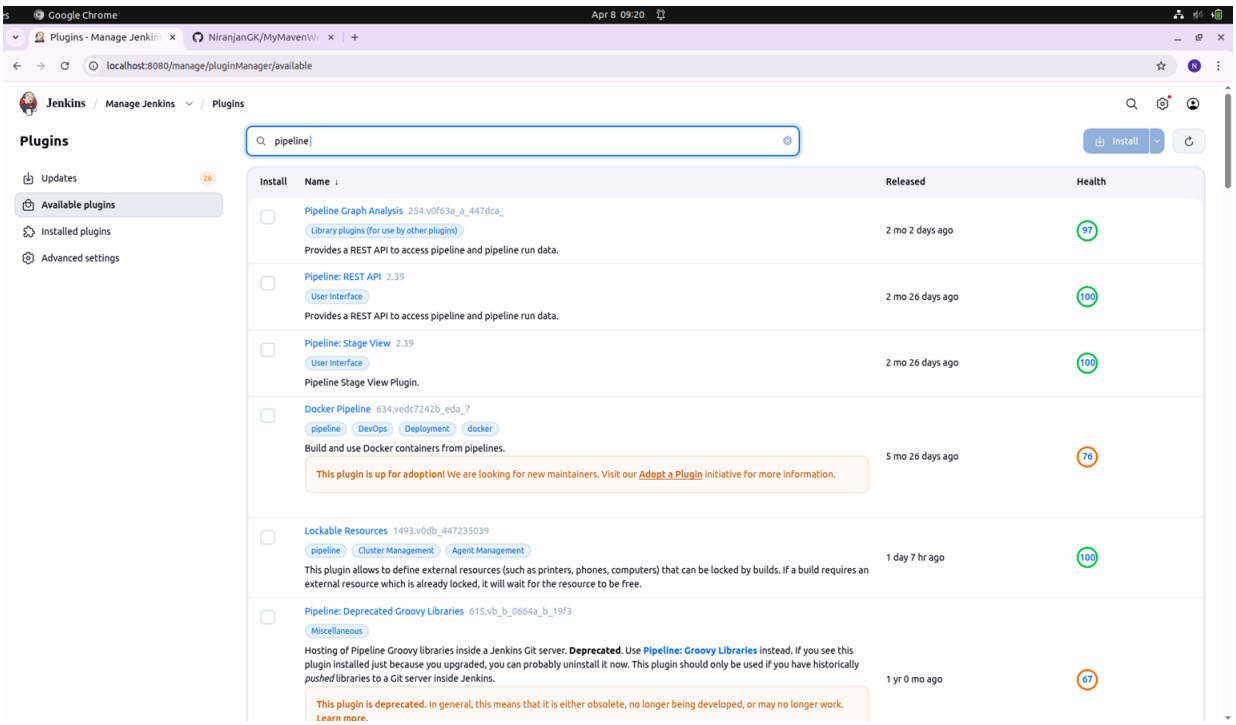


2. Navigate to:
   Manage Jenkins → Plugins





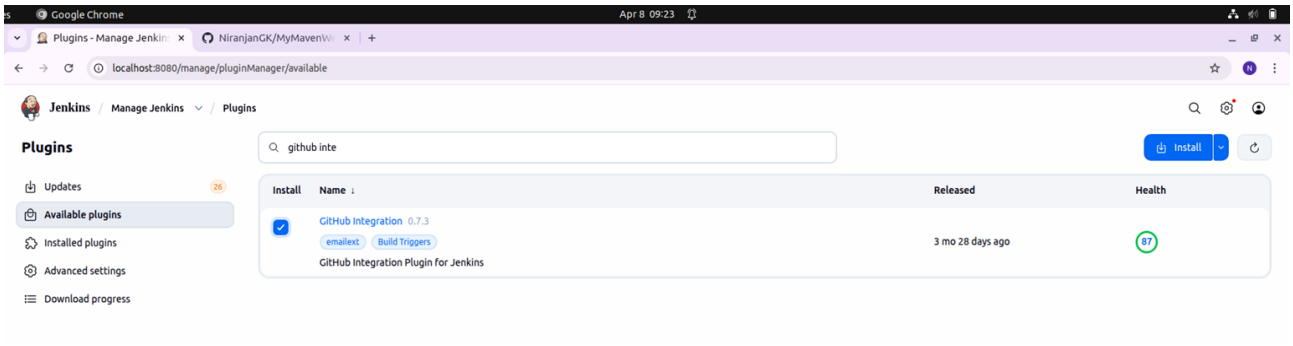
3. Go to the **Available Plugins** tab



4. Search for required plugins
5. Select plugins and click:
   Install without restart

5. Installed Plugins

5.1 GitHub Integration Plugin



Purpose

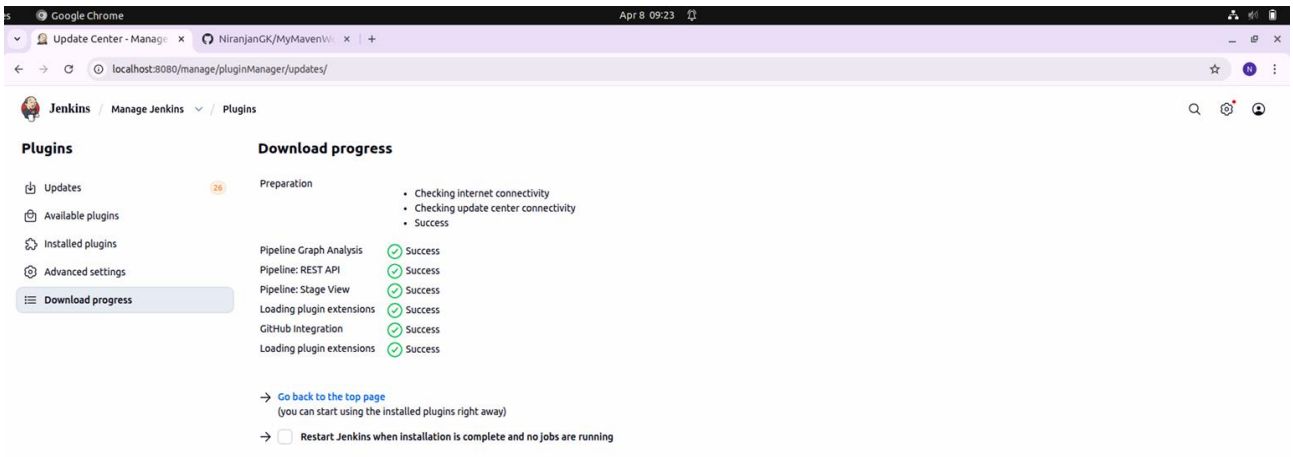
This plugin connects Jenkins with GitHub repositories, enabling automatic code checkout and webhook-based triggers.

Features

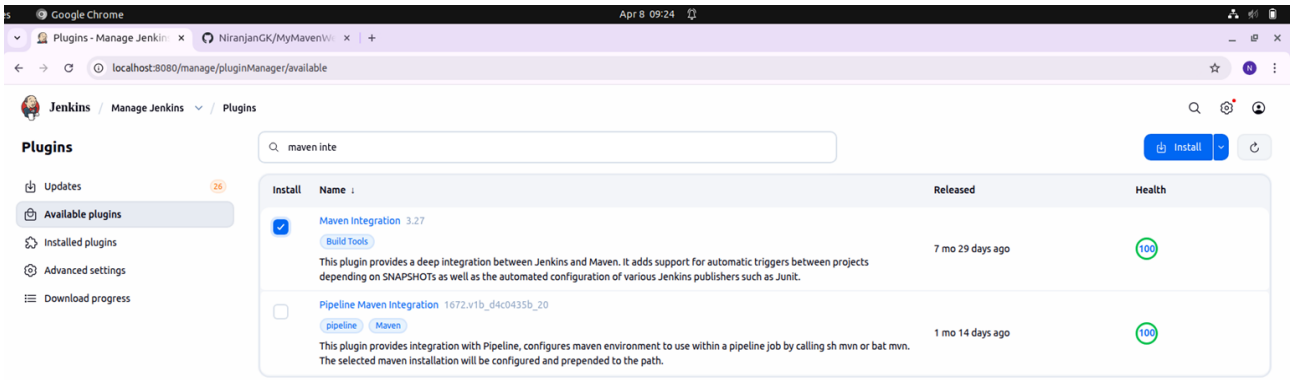
- ❿ Pull code from GitHub repositories
- ❿ Trigger builds on code changes
- ❿ Secure authentication using credentials

Configuration

- ❿ Add GitHub credentials in Jenkins
- ❿ Provide repository URL in pipeline job



5.2 Maven Integration Plugin



Purpose

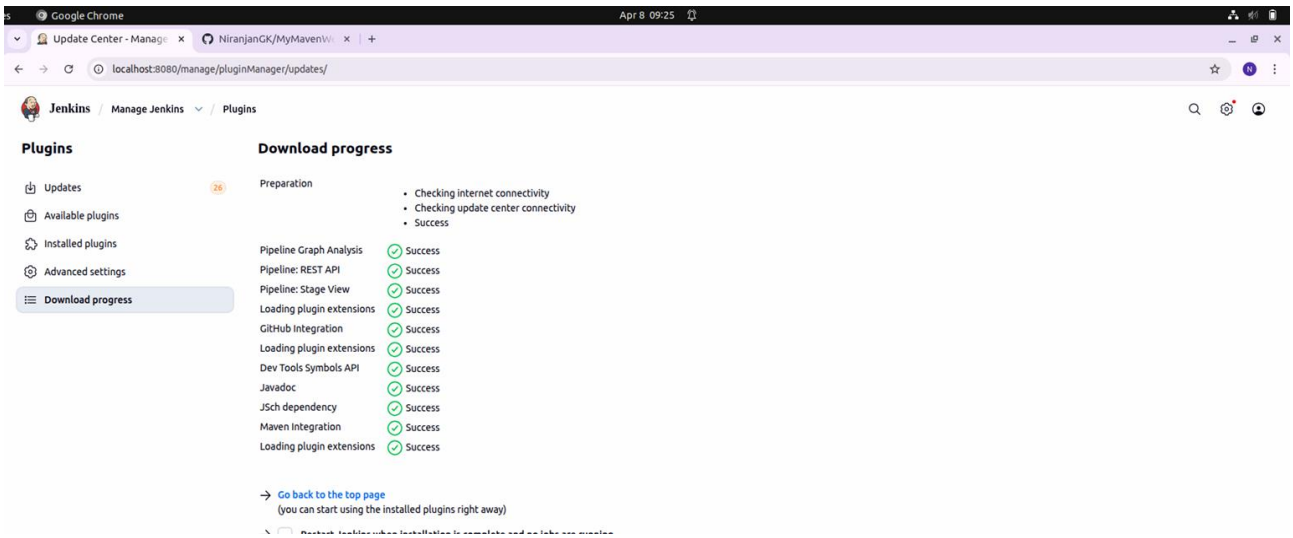
This plugin allows Jenkins to build Java applications using Maven.

Features

- ❿ Automates build lifecycle (compile, test, package)
- ❿ Supports dependency management
- ❿ Integrates with Jenkins pipeline

Configuration

- ❿ Go to **Manage Jenkins → Global Tool Configuration**
- ❿ Add Maven installation
- ❿ Provide name (e.g., Maven)



5.3 Pipeline Stage View Plugin

Purpose

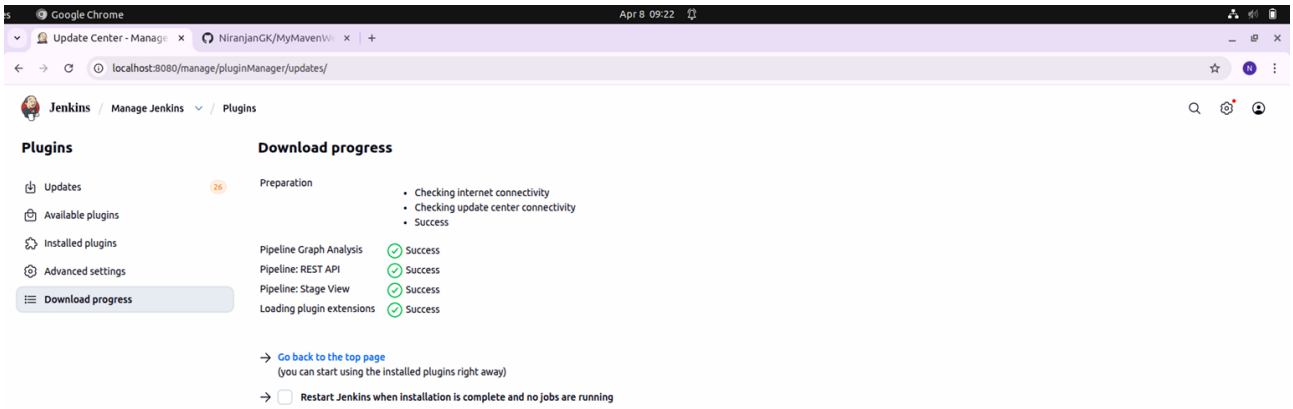
This plugin provides a visual representation of pipeline stages in Jenkins.

Features

- ❿ Displays each stage (Build, Test, Deploy)
- ❿ Shows success/failure status
- ❿ Helps in debugging pipeline errors

Usage

- ❿ Automatically appears when using Pipeline jobs
- ❿ Shows graphical stage view after build execution



5.4 Gradle Plugin

Purpose

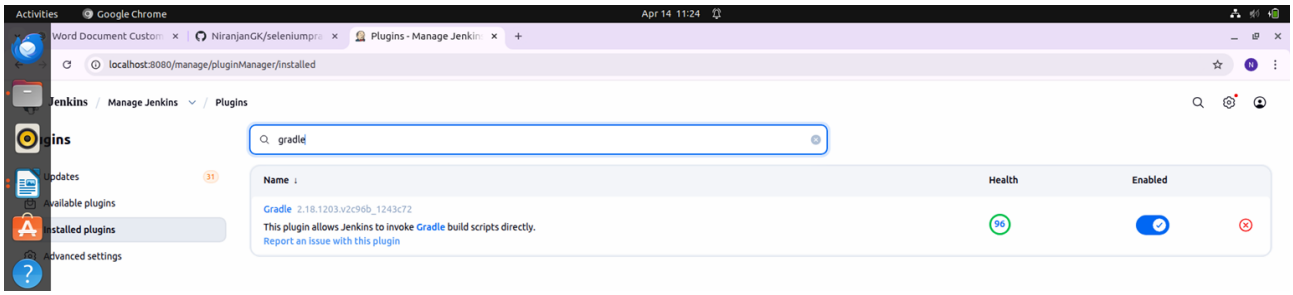
This plugin enables Jenkins to build and manage projects using the Gradle build automation tool.

Features

Displays execution of Gradle tasks (Build, Test, Run)
Supports automated build and dependency management
Shows build success/failure status
Helps in identifying build and test errors

Usage

Used in Jenkins pipelines by defining Gradle in the tools section
Executes Gradle commands like `gradle build`, `gradle test`, `gradle run`
Works after configuring Gradle in Global Tool Configuration



6. Result

- ❿ Required plugins installed successfully
- ❿ Jenkins is now integrated with GitHub and Maven
- ❿ Pipeline visualization enabled using Stage View
- ❿ System ready for CI/CD pipeline execution

7. Conclusion

The installation of essential Jenkins plugins enhances its capabilities and allows seamless integration with development tools. These plugins play a critical role in automating build, test, and deployment processes, making Jenkins a powerful CI/CD platform.